

# PubTrail - Firestore Database Schema

---

This document outlines the Firestore NoSQL database schema for **PubTrail**. The database is structured using root collections and nested subcollections to ensure efficient querying, real-time collaboration, and to avoid exceeding Firestore's 1MB document size limits.

## 0. High Level Schema

### Firestore Database Root

```
├── users/ (Collection)
│   ├── {userID} (Document)
│   │   ├── email: string
│   │   ├── avatarUrl: string
│   │   └── activeCrawl: Reference</crawls/{crawlUID}> | null
│   └── crawls/ (Collection)
│       ├── {crawlUID} (Document)
│       │   ├── crawlName: string
│       │   ├── createdBy: Reference</users/{userID}>
│       │   ├── lastStopLat: number | null
│       │   ├── lastStopLng: number | null
│       │   ├── isActive: boolean
│       │   ├── isPublic: boolean
│       │   ├── participants: array [Reference</users/{userID}>]
│       │   └── stops/ (Subcollection)
│       │       ├── {stopUID} (Document)
│       │       │   ├── pubName: string
│       │       │   ├── coordinates: GeoPoint [lat, lng]
│       │       │   ├── notes: string
│       │       │   └── visitedOn: Timestamp
│       │       └── media/ (Subcollection)
│       │           ├── {mediaUID} (Document)
│       │           │   ├── url: string
│       │           │   └── uploadedBy: Reference</users/{userID}>
│       └── media/ (Subcollection)
│           ├── {mediaUID} (Document)
│           │   ├── url: string
│           │   └── uploadedBy: Reference</users/{userID}>
```

---

## 1. Root Collection: **users**

This collection stores the profiles of all users registered on the application. While Firebase Auth Users are used by the app, each sign-in/up should cause a respective **users** document to be created. This allows us to

fetch arbitrary user information, which is not allowed by Firebase itself (it only allows accessing the logged-in user information).

- **Document ID:** `userUID` (Generated by Firebase Authentication)
  - **Fields:**
    - `email` (`String`): The user's email address.
    - `avatarUrl` (`String`): A URL pointing to the user's profile picture.
    - `activeCrawl` (`Reference<crawls> | Null`): A reference to the active crawl this user is participating in.
- 

## 2. Root Collection: `crawls`

This collection stores the top-level "header" information for every pub crawl session. Keeping this data separate from the stops allows users to quickly fetch their crawl history or discover nearby public crawls.

- **Document ID:** `crawlUID` (Auto-generated)
- **Fields:**
  - `crawlName` (`String`): The title of the pub crawl.
  - `createdBy` (`Reference<users>`): A reference to the user who created the crawl (e.g., `/users/userUID`).
  - `lastStopLat` (`Number | Null`): Latitude of the latest stop in this crawl, or `null` when no stop coordinates are available.
  - `lastStopLng` (`Number | Null`): Longitude of the latest stop in this crawl, or `null` when no stop coordinates are available.

The `lastStopLat` and `lastStopLng` are used instead of a single `GeoPoint` field, because there is no way to query on individual `GeoPoint` field coordinates. Another approach would be to store a **geohash** string instead, and query using the `geofire-js` library.

- `isActive` (`Boolean`): A flag indicating whether this crawl is still active or has been ended.
  - `isPublic` (`Boolean`): A flag indicating whether this crawl should be visible on the public map.
  - `participants` (`Reference<users>[]`): An array containing references to the users that have participated (at any point) in the crawl. Joining adds a user's reference to this array.
- 

### 2.1 Subcollection: `stops`

Nested inside each `crawlUID` document, this subcollection tracks the individual pubs visited during that specific crawl.

- **Path:** `/crawls/{crawlUID}/stops/{stopUID}`
  - **Document ID:** `stopUID` (Auto-generated)
  - **Fields:**
    - `pubName` (`String`): The name of the pub visited.
    - `coordinates` (`GeoPoint`): The precise latitude and longitude of the pub.
    - `notes` (`String`): Text documenting the group's experience at this stop.
    - `visitedOn` (`Timestamp`): The exact date and time the group arrived at this stop.
-

## 2.2 Subcollection: **media**

Nested inside each **crawlUID** document, this subcollection stores the photos uploaded for a crawl.

- **Path:** `/crawls/{crawlUID}/media/{mediaUID}`
- **Document ID:** **mediaUID** (Auto-generated)
- **Fields:**
  - **url** (**String**): The direct URL to the image.
  - **uploadedBy** (**Reference<users>**): A reference to the user who uploaded the photo (e.g., `/users/userUID`).